

Lab of Web Technologies

Comprehensive Study Guide & Expanded Mock Exam

Covers all 15 lectures: the Internet & WWW, HTML, CSS, JavaScript, jQuery / Bootstrap / AJAX, HTTP & Flask, Sessions / Cookies / Authentication, and Databases & SQL.

Prepared for: **Alessio Manera** · Group-11 (solo group)

Project: Museum Booking and Info Web App

Target exam session: **September 2026**

Course: Lab of Web Technologies · Ca' Foscari University of Venice · Prof. Zeynep Yücel · A.Y.
2025/2026

Generated 28 June 2026

Table of Contents

Table of Contents	2
Part 0 — Read This First: Exam Strategy and Logistics	5
Straight talk about this exam	5
Your project, in one paragraph	5
Timeline (counts backward from the official September exam date)	5
Part 1 — The Internet and the World Wide Web	7
1.1 The Internet	7
1.2 The World Wide Web (and why it is NOT the Internet)	7
1.3 Anatomy of a URL request (host, server, resource)	8
1.4 URI vs. URL vs. URN	8
1.5 History — how we got here (names and years come up in MC questions)	8
1.6 Web 1.0 vs Web 2.0 vs Web 3.0	9
Part 2 — HTML	10
2.1 What kind of language is HTML?	10
2.2 Structure vs. Presentation — why we separate them	10
2.3 The basic HTML template	10
2.4 Content flow, block vs. inline elements	11
2.5 Text structuring elements	11
2.6 Semantic vs. non-semantic tags	13
2.7 Links and URLs	13
2.8 Images	14
2.9 The (head) and its key children	15
2.10 The "Discover Italy!" assignment (context, not exam content)	15
Part 3 — CSS	16
3.1 What CSS is and who maintains it	16
3.2 Three ways to attach CSS to a document	16
3.3 CSS rule anatomy	16
3.4 Selectors	17

3.5 Specificity — the single most exam-tested CSS topic	18
3.6 Box model and positioning	18
3.7 Color, fonts, and units	19
Part 4 — JavaScript	21
4.1 What JavaScript is	21
4.2 Syntax basics	21
4.3 Data types and values	21
4.4 Operators	22
4.5 Expressions, statements, control flow	24
4.6 Functions	24
4.7 Loading JavaScript and the window/DOM model	25
4.8 HTML Forms in JavaScript	26
Part 5 — jQuery, Bootstrap and AJAX	27
5.1 Why third-party libraries exist	27
5.2 jQuery	27
5.3 Bootstrap	28
5.4 AJAX and asynchronous programming	29
Part 6 — HTTP, Web Servers and Flask	31
6.1 HTTP, the protocol underneath everything	31
6.2 URL anatomy (the 4-part practical view used for HTTP)	31
6.3 Request line, status line, headers	31
6.4 HTTP methods — the most exam-relevant table in this whole course	31
6.5 Web servers and frameworks	32
6.6 Flask essentials — this is your project's backbone	33
Part 7 — Sessions, Cookies and Authentication	36
7.1 From stateless to stateful	36
7.2 What a cookie is and what it is for	36
7.3 Cookie types and the attributes that control them	36
7.4 Security vs. privacy — two different concerns	37
7.5 Authentication	37
7.6 HTTPS = HTTP over SSL/TLS	38

7.7 Cookie attack categories (named threats — good MC/open material)	38
7.8 Cookies in Flask (practical syntax for your project)	38
Part 8 — Databases and SQL	40
8.1 Why a DBMS instead of a plain file	40
8.2 Concrete vs. abstract knowledge (the conceptual modeling step)	40
8.3 From knowledge to tables	40
8.4 Database and DBMS definitions	41
8.5 SQL: a declarative language with 4 sublanguages	41
8.6 SQLite, prepared statements, and password hashing — the Python side	42
8.7 The "Cookbook App" exercise (context for hands-on practice)	43
Part 9 — Quick-Reference Cheat Sheets	44
CS-1 — CSS Specificity Calculator	44
CS-2 — HTTP Methods at a Glance	44
CS-3 — Cookie Attributes	44
CS-4 — JavaScript Falsy Values (memorize the full list)	44
CS-5 — Block vs. Inline HTML Elements	45
CS-6 — TCP/IP Five Layers, Top to Bottom	45
CS-7 — CSS Selector Combinators	45
CS-8 — Flask Quickstart Skeleton	45
CS-9 — SQL in Four Lines	45
Part 10 — Mock Exam (Expanded Practice Set)	46
Section A — Multiple Choice (Questions 1–10)	46
Section B — Open-Ended Questions (Questions 11–16)	49

Part 0 — Read This First: Exam Strategy & Logistics

Straight talk about this exam

This is a study guide built directly from your course slides and your professor's own guidelines document, organized into one reference. Here is the honest read on how to use it well.

- The written exam is **strictly based on the slides** (the professor said so explicitly). That means precise wording, exact numbers, and exact definitions matter more than general understanding. "I get the gist" is not the same as "I can reproduce the specificity formula or the five TCP/IP layers in order." Memorize the tables in this guide, not just the concepts.
- The format is fixed: **5 multiple-choice + 3 open-ended questions**, each half worth 30+1 points, and you need **at least 18/30 on both halves** (written exam AND project/oral) to pass. A great project does not save you from a weak written score, and vice versa — they are graded independently. Do not let project work crowd out written-exam revision in the final weeks.
- You are a **single-person group (Group-11)**. That is a real risk multiplier: no teammate to catch a missed deadline, split the documentation, or sanity-check your Flask routes the night before submission. Build in personal checkpoints well before the official ones.
- September is the **last exam session of the academic year**. If you fail or skip either half, you lose any grade you banked and must redo **both** the written and the project/oral the following year. There is no "keep the project grade, retake just the written" option. Treat both halves as equally non-negotiable.
- You cannot sit this exam at all without having already passed **"Introduction to Coding and Data Management."** Confirm that box is ticked before you invest more time here.

Mentor's take — Open-ended questions on an exam like this tend to reward the kind of answer a professor could lift a sentence from their own slide to grade against. Don't write vague prose about "the internet is important" — name the mechanism (e.g. "specificity is calculated as four counters: inline, IDs, classes, elements") and you will pick up marks a fuzzy answer won't.

Your project, in one paragraph

Topic confirmed with Professor Yucel: a **Museum Booking and Info Web App** — ticket booking for museums/cultural sites, with AI-assisted guidance and personalized recommendations. Required stack: HTML (structure), CSS (presentation), JavaScript (behavior), Python + Flask (server). The app needs multiple static and dynamic pages and a server able to handle different HTTP request types. Everything in Parts 6–8 of this guide (Flask, routing, Jinja2, sessions/cookies, SQL) is directly reusable for that project — it is not just exam trivia.

Timeline (counts backward from the official September exam date)

Milestone	Deadline	Status / Note
Submit group members	4 weeks before exam	Done — registered as Group-11
Submit 1-page project proposal	3 weeks before exam	Upload to Moodle
Receive professor feedback	2 weeks before exam	Revise & resubmit if requested
Submit final project archive	1 week before exam	Late submissions are not considered
Oral presentation	Exam day	Purpose, features, strengths, limitations

- Final submission = an archive (.zip / .tar.gz) on Moodle containing the project AND a document with: (1) project description & goals, (2) instructions to run the app, (3) a clear description of your role and contributions.
- Because you are solo, "your role and contributions" is simply everything — write it up cleanly anyway, it is part of the grading rubric, not a formality.

Part 1 — The Internet & the World Wide Web

1.1 The Internet

The **Internet** is the global system of interconnected computer networks that use the **Internet Protocol Suite (TCP/IP)** to link devices worldwide. It is "global" because it merges private, public, academic, business, and government networks into one interconnected whole.

- A **computer network** lets nodes (computers) share resources / exchange data.
- Each node is identified by an **address** and is generally called a **host**.
- Two hosts can exchange data even without a direct connection between them, thanks to different network topologies.
- A **communication protocol** is a set of rules for exchanging information over a network. Protocols are layered in a **stack**: each layer leverages the layer below it; the lowest layer controls the hardware that physically sends information across the medium.

TCP/IP is a **conceptual model** for all protocols used on the Internet, named after its two fundamental protocols, TCP and IP. It specifies how data should be **packetized, addressed, transmitted, routed, and received**.

The five TCP/IP layers (bottom to top) — exam favorite, know the order and one job per layer

Layer	Responsible for
1. Physical	Physical transmission of bits over cables, connectors, hardware (e.g. Ethernet, Wi-Fi).
2. Data Link	Packing data into frames, transferring frames between two nodes, error detection/correction.
3. Network	Routing packets to their destination; addressing; flow control & congestion avoidance.
4. Transport	Reliable end-to-end communication; flow control; error checking/correction (e.g. TCP).
5. Application	Services that use lower layers: email, web browsing, file transfer, streaming; also handles authentication/encryption.

Mentor's take — A classic trick MC question swaps the order or swaps two job descriptions between layers (e.g. attributing "routing" to Data Link instead of Network). Drill the order: Physical → Data Link → Network → Transport → Application.

1.2 The World Wide Web (and why it is NOT the Internet)

The **WWW** is an information system in which resources are identified by **Uniform Resource Locators (URLs)**, linked by hypertext, and accessible over the Internet via a **web browser**. The Internet is the network; the Web is one application that runs on top of it.

	Internet	World Wide Web
What it is	Vast network of interconnected computers/servers using TCP/IP.	System of interlinked hypertext documents & multimedia accessed via the Internet.
Functionality	Email, file transfer, online gaming, social networking, etc.	Web pages built with HTML: text, images, video, links.
Infrastructure / Protocols	Routers, cables, servers, global data transmission.	Primarily HTTP and HTTPS.

- Architectural bases of the WWW: URLs identify resources; web agents communicate via standardized protocols with defined syntax/semantics; resources have a representation a browser can interpret and visualize.

1.3 Anatomy of a URL request (host, server, resource)

Example: `http://www.example.org/home.html`

- Host name** — resolved to an IP address (e.g. 203.0.113.4) via the globally distributed **Domain Name System (DNS)**.
- Server** — the browser opens a **TCP connection** to the server and talks HTTP.
- Resource** — the browser asks for a named resource (here, `home.html`). A resource is **not necessarily a file** — it can be generated on-the-fly by the server. Each resource has an associated representation, its **MIME type** (e.g. `text/html`: "text" is the type, "html" the subtype).

A typical HTTP transaction: the browser sends a request, the server replies with a response that includes a **status code**. 200 OK = success. Other common codes: 302 Redirect (go elsewhere), 404 Not Found.

1.4 URI vs. URL vs. URN

Term	Definition	Example
URI	General umbrella term: a string identifying a resource by location, name, or both.	<code>http://example.com</code> , <code>urn:isbn:0451450523</code>
URL	A URI that both identifies AND locates a resource (access mechanism + network location).	<code>https://www.unive.it</code>
URN	A URI that names a resource persistently/independently of location (no access info).	<code>urn:isbn:3959401035</code>

1.5 History — how we got here (names & years come up in MC questions)

Year	Event
1989	Tim Berners-Lee (at CERN) writes "Information Management: A Proposal" — the seed of the Web.
Oct. 1990	He completes the three foundational technologies: HTML, URI (URL), HTTP.
End of 1990	First web page served on the open Internet.
1991	People outside CERN invited to join the web community.
1993	NCSA Mosaic released — first browser to show text and images together on one page.
1994-1996	Internet Explorer, Netscape Navigator, Opera born.
1995	JavaScript introduced (Netscape Navigator 2; idea by Marc Andreessen) — lets pages be dynamic.
1996	CSS born — separates content from presentation.
2004	Tim O'Reilly popularizes "Web 2.0".
2005	Jesse James Garrett publishes the "Ajax" paper; IRI defined.

Tim Berners-Lee built HTML/URI/HTTP while trying to ease collaboration among researchers at CERN. His original proposal was not immediately accepted, but his boss gave him time to develop it.

1.6 Web 1.0 vs Web 2.0 vs Web 3.0

Era	Defining trait
Web 1.0 (~1989–2004)	Static pages. Content made by a few "publishers", consumed by many readers.
Web 2.0 (popularized 2004)	Dynamic, user-generated content; participatory/social web; ease of use; interoperability across products/devices (e.g. social media). Not a technical standard, just a design philosophy.
Web 3.0 / Semantic Web	Aims to make Internet data machine-readable , so computers can analyze content/links/transactions directly. Beyond client-server, toward a peer-distributed system. Still not a standard, still under active research.

Mentor's take — This exact question — "what does Web 3.0 aim to achieve?" — is the first question on your official sample exam. The answer is "make Internet data machine-readable," not "build faster browsers" or "improve HTML." Don't overthink it; the slide phrase is the answer.

Part 2 — HTML

2.1 What kind of language is HTML?

Category	Trait	Examples
Programming	Needs compiling before executing; one-shot version.	C, C++, Java, C#
Scripting	Interpreted, no explicit compilation; designed for run-time systems.	Python, Perl, Ruby, PHP
Markup	Interpreted by the browser; for annotating documents (structure, not logic).	HTML, XML, Markdown

- A **markup language** annotates a document with **tags** that add meaning/structure to text. It focuses on structure and leaves visual presentation to the interpreter (the browser).
- HTML specifies: which tags exist, how they should be used, and what attributes they can carry.
- HTML is **declarative**: it only defines structure — pages built with HTML alone are not interactive.

2.2 Structure vs. Presentation — why we separate them

Structure = the parts authors create (titles, paragraphs, lists). **Presentation** = how those parts are actually displayed on a device (screen, printer, etc.). Three slide-stated reasons to keep them apart:

1. Authors (programmers) and designers have different skills — let authors write content/structure and designers control appearance; designers can enforce a consistent style across an organization.
2. If markup is pure content+structure, appearance can change fast: different presentations for different media (screen vs print vs aural device) and the whole site's look can change without touching the content.
3. Accessibility improves — the same structured document can be read by a Braille printer (limited vision) or a text/screen reader (limited hearing), because presentation is not baked into the content.

Mentor's take — "Explain why we separate structure from presentation" is one of the three open questions on your official sample exam. Memorize those three numbered reasons verbatim — they map exactly to slide 9/26 of Lecture 2, and an exam grader will be checking for them.

2.3 The basic HTML template

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8" />
</head>
<body>
Hello world
</body>
</html>
```

- `<!DOCTYPE html>` is NOT a tag — it declares the HTML version (HTML5 uses this exact short form; HTML 4.01 needed a long PUBLIC/DTD string).

- Every document has 3 document-level tags: `<html>`, `<head>`, `<body>`.
- `<head>` holds meta-information (title, charset, CSS/JS links). `<body>` holds the actual displayed content.
- Elements nest into a **tree**; every element except `<html>` has exactly one parent (e.g. parent of `<meta>` is `<head>`; parent of `<head>` is `<html>`).
- Attributes appear as `attribute="value"`. Boolean attributes appear alone, e.g. `<input read-only />`.

Four rules of well-formatted HTML (classic "spot the error" MC bait)

1. Begin with the DOCTYPE declaration.
2. Tags close in the **opposite order** they were opened. Wrong: `<p> hello <strong> world </p> </strong>` — Right: `<p> hello <strong> world </strong></p>`.
3. All non-empty elements must be terminated (closed).
4. All attribute values should be quoted: `type="checkbox"`, not `type=checkbox`.

White space inside HTML is condensed by the browser to a single space wherever line breaks or multiple spaces appear — text formatting can only be done via tags (or `<pre>` to preserve literal formatting, which switches to a monospace font by default).

2.4 Content flow, block vs. inline elements

Content flow = how elements are organized/rendered relative to each other: either **sequential ordering** (first in the code = first on the page) or **stacking order** (when elements nest, the innermost sits in front, the outermost sits in back).

Block elements	Inline elements
Change the content flow.	Do NOT change the content flow.
Separated by line breaks + vertical space.	Not separated from each other.
Occupy full available width + needed height.	Take only as much width as the content needs.
Can contain block, text, or inline elements.	Always found inside a block element.
Cannot be placed inside an inline element.	—

- **Common block elements:** `div`, `h1–h6`, `p`, `ol`, `ul`, `li`, `nav`, `form`, `table`.
- **Common inline elements:** `span`, `a`, `img`, `em`, `b`, `i`, `br`, `small`, `button`.

2.5 Text structuring elements

- `<p>` — paragraph; controls line spacing within and between paragraphs.
- `<pre>` — preserves whitespace/newlines literally; uses a monospace font.
- `<h1>...<h6>` — 6 heading levels, `h1` = most important; each acts like a paragraph (automatic line break + extra spacing before/after).
- `<br>` — manual line break without ending the current paragraph.

Lists

- `<ol>` ordered list, `<ul>` unordered list — both contain `<li>` items only; a list item is itself a block element and may contain another nested list.

- `<dl>` definition list — pairs of `<dt>` (term) and `<dd>` (description). No restriction on what else can appear inside `<dt>/<dd>`.

```
<dl>
  <dt>Equilateral triangle</dt>
  <dd>3 equal sides and 3 equal 60-degree angles.</dd>
  <dt>Isosceles triangle</dt>
  <dd>2 equal sides and 2 equal angles.</dd>
</dl>
```

Tables

Built from `<table>`, with rows `<tr>`, header cells `<th>`, and data cells `<td>`.

```
<table>
  <tr><th>header-1</th><th>header-2</th></tr>
  <tr><td>cell-1-1</td><td>cell-1-2</td></tr>
</table>
```

Quotations

`<blockquote cite="URL">` wraps a block-level quotation. The `cite` attribute renders as nothing special in browsers, but search engines can use it; it is good practice to always cite a quotation's source.

2.6 Semantic vs. non-semantic tags

A **semantic element** clearly describes its own meaning to both browser and developer (e.g. `<table>` says what it contains; `<div>` says nothing).

Semantic tags	Meaning
<code>em</code> / <code>strong</code>	Emphasis (italics) / stronger emphasis (bold).
<code>abbr</code>	An abbreviation, e.g. <code><abbr>CA</abbr></code> for California.
<code>cite</code>	Cited title of a work (default italics).
<code>code</code>	A code sample, usually fixed-width font.
<code>dfn</code>	A definition.
<code>header</code> / <code>nav</code> / <code>article</code> / <code>section</code> / <code>footer</code>	Page regions: intro/logo/nav; nav links; self-contained content (e.g. blog post); thematic grouping; footer info.
<code>figure</code> / <code>figcaption</code>	Self-contained content with a caption (e.g. an illustration) / its caption.
<code>time</code> / <code>address</code>	A specific point/period in time / contact info for the document's author.

Non-semantic / deprecated tags	Note
<code>div</code> / <code>span</code>	Generic block / inline container — purely for styling or grouping, no meaning.
<code>b</code> / <code>i</code> / <code>u</code>	Bold / italic / underline with no implied importance or emphasis.
<code>font</code> , <code>center</code>	Deprecated in HTML5 — used to set font/colour or center text directly.
<code>link</code> (inside head)	Has a real purpose (linking a stylesheet) but conveys no content meaning.

`<span>` has zero inherent visual effect — it exists purely to be targeted by CSS or scripts, exactly like `<div>` but for inline text.

2.7 Links and URLs

The anchor tag `<a></a>` needs at least an `href` attribute to be useful: `<a href="spotdata.html">Spot</a>`.

The 9-part general URL anatomy (slide-level detail — good MC material)

```
<scheme>://<user>:<password>@<host>:<port>/<path>;<params>?<query>#<frag>
```

Part	Meaning
scheme	Protocol to retrieve the resource: http, https, ftp, rtsp...
user:password	Optional credentials to authenticate with the server (default user: "anonymous").
host	Hostname (resolved via DNS) or IP address. Always required.
port	TCP port the server listens on. Can be omitted — defaults apply (HTTP default = 80).
path	The local resource name; segments separated by "/" (e.g. seg1/seg2/seg3).
params	List of name-value pairs attached to a path segment, extra info for the server.
query	Restricts the context of a resource, e.g. ?item=1234&num=3 (pairs joined with "&").
frag	A fragment/anchor identifying a part inside the resource (e.g. a specific paragraph).

Absolute vs. relative URLs

- **Absolute URL** — contains everything needed to reach the resource.
- **Relative URL** — just the resource name; needs a **base URL**, obtained either via an explicit `<base href="..." />` tag in the head, or (more common) by using the absolute URL of the page that contains the relative link.
- Example: page at `http://www.joes-hardware.com/tools.html` links to `./hammers.html` → resolves to `http://www.joes-hardware.com/hammers.html`.

URL escaping & internationalization

- HTTP URLs may only contain standard **ASCII** characters. Non-ASCII or reserved characters must be **escaped** as % + a 2-digit hex code (default charset UTF-8), e.g. # → %23, @ → %40.
- **IRI (Internationalized Resource Identifier, 2005)** extends URLs with full Unicode, staying backward-compatible via standard escaping. Browsers built for IRI render the real characters.
- Risk: the **IDN homograph attack** — using look-alike Unicode characters to spoof a domain, e.g. `paypαl.it` instead of `paypal.it`.

2.8 Images

`` — `src` (image URL) and `alt` (fallback / accessibility text for screen readers) are both required. Images carry no text content of their own, so the `<img>` tag must sit inside a block element such as `<p>`.

	Raster images	Vector images
Built from	Pixels (color squares).	Mathematical equations, lines, curves.
Scaling	Loses quality when zoomed/enlarged.	Scales up/down with no quality loss.
File size	Generally larger; can slow page loads.	Lightweight (just formulas).
Typical use	Photos, detailed digital art.	Logos that must fit many sizes (card → billboard).
Formats	.jpeg, .gif, .png	.svg, .eps, .emf

Format	Colors	Compression	File size	Best for
Raw	Billions	No	Very big	Editing
JPG	16.1M	Lossy	Small	Websites, storage
GIF	256	Lossless	Small	Animation
PNG	16.1M + transparency	Lossless	Big	Websites, editing, storage
TIFF	Variable	Variable	Big	Editing, printing
BMP	Variable	Lossless	Big	Windows apps

2.9 The <head> and its key children

- `<title>` — required once; sets the browser title-bar text. A missing close tag can stop the document from loading in many browsers.
- `<meta>` — sets values equivalent to HTTP response headers, most commonly the document's character set.
- `<base href="...">` — sets an absolute base URL used to resolve every relative link in the document; usually placed right after `<title>` since it can affect subsequent `<script>`, `<link>`, `<style>` tags.
- `<link rel="stylesheet" ...>` — declares a relationship to another document, most commonly a CSS stylesheet.

Common & custom attributes

- `id` — unique identifier referenced by CSS/scripts.
- `class` — names a semantic group the element belongs to.
- `style` — inline CSS rules for that element only.
- `hidden` — tells the user agent to hide the element's content.
- **Custom data- * attributes** (HTML5): must start with `data-` plus at least one more character, e.g. `data-borrower="Vonnegut, K."` — used to expose extra info to client-side scripts.

2.10 The "Discover Italy!" assignment (context, not exam content)

Lecture 4's practical assignment: a travel-agency site with four linked pages — Home (description + image), Travel plans (a table of plans, each linking to a dedicated page with picture/description/destinations/price), Contact us (a mail form), and Tourism in Italy (image + quoted text). This is pure practice; the exam tests the underlying HTML concepts above, not this specific content.

Part 3 — CSS

3.1 What CSS is and who maintains it

CSS (Cascading Style Sheets) describes how HTML elements are displayed (screen, paper, other media). It saves work by controlling the layout of many pages at once and adapting design across devices. CSS is created/maintained by the **CSS Working Group** inside the **W3C**; ratified specs are called **recommendations** (the W3C does not control actual implementations).

Version	Year	What it added
CSS1	Dec. 1996	The language itself + a simple visual formatting model for all HTML tags.
CSS2	May 1998	Media-specific stylesheets (printers, aural devices), downloadable fonts, positioning, tables.
CSS3	2011	NOT one monolithic spec — split into independent modules (selectors, text effects, etc.).

This is the official sample-exam answer for "what is the purpose of CSS": "to separate content from presentation." If a question lists distractors like "add interactivity" (that's JS) or "define document structure" (that's HTML), don't take the bait.

3.2 Three ways to attach CSS to a document

1. `<link>` tag → external .css file. `<link rel="stylesheet" type="text/css" href="sheet1.css" media="all"/>`. The media attribute targets a medium: all, print, screen, aural, embossed, projection, tv.
2. `<style>` element inside `<head>` — an embedded stylesheet (same effect as an external file, just inline in the HTML).
3. style attribute directly on a tag — affects only that one element. **The slides explicitly say this is generally NOT recommended.**

You can link **multiple stylesheets** to one document — the browser loads all of them, combines their rules, and applies them together. This is exactly why it is called "**cascading**": multiple sources of styling (head, external file, inline element) flow together into one virtual stylesheet, resolved by specific cascade rules.

Replaced vs. non-replaced elements

- **Replaced elements** — content is replaced by something not directly represented in the document, e.g. `<img>`, `<input>`.
- **Non-replaced elements** — presented inside a box generated by the element itself; this is the majority of elements.

3.3 CSS rule anatomy

```
h1 {  
  color: red;  
  background: yellow;  
}
```


- `h1` is the **selector**; everything inside `{ }` is the **declaration block**, made of one or more **declarations** (a property : value pair, always ending in a semicolon).
- Comments use `/* ... */` and can span multiple lines.

3.4 Selectors

Selector type	Syntax / example	Matches
Element	<code>h1 {color: gray;}</code>	Every element of that type. (Group with commas: <code>h2, p {...}</code>)
Universal	<code>* {color: red;}</code>	Every element in the document.
Class	<code>.warning {font-weight:bold;}</code>	Elements with <code>class="warning"</code> . Combine with an element: <code>p.warning {...}</code> only matches <code><p class="warning"></code> .
Multiple classes	<code>.medium.red {...}</code>	Elements that carry BOTH classes, in any order, e.g. <code>class="red medium"</code> .
ID	<code>#para1 {color:red;}</code>	The single element with that id. IDs should be unique per page.
Descendant	<code>div p {color:red;}</code>	Any <code><p></code> that is a descendant of a <code><div></code> , at any depth.
Child	<code>body > p {...}</code>	Only <code><p></code> that are a DIRECT child of <code><body></code> .
Pseudo-class	<code>a:hover {...}</code>	A "phantom" state: <code>a:link</code> , <code>a:visited</code> , <code>a:hover</code> , <code>a:active</code> .
Attribute	<code>p[lang="fr"] {...}</code>	<code>p[lang]</code> = has the attribute; <code>p[lang="fr"]</code> = exact match; <code>p[lang~="fr"]</code> = "fr" appears as one word in a space-separated value.

Mentor's take — Watch for the difference between `div p` (descendant — any depth) and `body > p` (child — must be a direct child). Exam writers love swapping these in a "what does this selector match" MC question.

3.5 Specificity — the single most exam-tested CSS topic

When two or more rules could apply to the same element, the browser computes the **specificity** of each rule's selector and gives the win to the higher value. Specificity is four counters, computed like this:

1. Start at 0.
2. Add **1** for every **element** selector.
3. Add **10** for every **class**, pseudo-class, or attribute selector.
4. Add **100** for every **ID**.
5. Add **1000** for an inline `style="..."` attribute — this always wins.

Worked examples straight from the slides — memorize the pattern, not just the numbers

Selector	Specificity (inline, ID, class, element)
<code>h1 {color: red;}</code>	0, 0, 0, 1
<code>p em {color: purple;}</code>	0, 0, 0, 2
<code>.grape {color: purple;}</code>	0, 0, 1, 0
<code>*.bright {color: yellow;}</code>	0, 0, 1, 0
<code>p.bright em.dark {color: maroon;}</code>	0, 0, 2, 2
<code>#id216 {color: blue;}</code>	0, 1, 0, 0
<code><h3 style="color: blue;"></code>	1, 0, 0, 0

Inheritance and the "no specificity" gotcha

Inheritance applies some styles not just to a targeted element but to its descendants too (e.g. `h1 {color: gray;}` also colors a nested `<em>`). Box-model properties are NOT inherited (to avoid chaos). **Crucial subtlety:** an inherited value has *no specificity at all* — *not even zero*. So a directly-applying rule with any real specificity beats inheritance, even from a high-specificity ancestor rule, which can produce surprising results (classic example: an `a:link` rule overriding an inherited `#toolbar` color on a link nested inside the toolbar — fixed by writing an explicit `#toolbar a:link {...}` rule).

Cascade tie-break + !important

- If two rules have **equal specificity**, the one declared **later** in the stylesheet/document wins.
- **Cascade algorithm, simplified:** (1) find all rules matching the element, (2) sort by specificity (higher wins), (3) if tied, sort by source order (later wins).
- **!important** overrides ALL other rules for that property on that element, regardless of specificity — use sparingly.

3.6 Box model and positioning

Every element generates one or more rectangular **element boxes**: a content area surrounded by optional **padding, border, outline, and margin** (in that order, content outward).

- `padding / padding-top/bottom/left/right` — space inside the border, around the content.
- `border-width / border-style` — thickness / line style of the border itself.

- `margin / margin-top/bottom/left/right` — blank space OUTSIDE the border.

The five values of the position property

Value	Behavior
static	Default; normal flow, box is not "positioned" at all.
relative	Offset from where it WOULD have been; original space is preserved (still reserved in flow).
absolute	Removed from normal flow entirely; positioned relative to its containing positioned block; original space collapses as if the element never existed.
fixed	Like absolute, but its containing block is the browser window/viewport itself.
sticky	Stays in normal flow until a scroll condition is met, then "sticks" in place.

`top/left/right/bottom` describe the offset from the corresponding side of the containing block (used with `relative`, `absolute`, `fixed`, `sticky`). Positive values push the element down/right from that side; negative values can push it outside its container.

- `overflow`: `visible` (default, renders outside the box), `hidden` (clipped), `scroll` (always shows scrollbars), `auto` (scrollbars only when needed).
- `background-clip`: `border-box` (default, extends behind border), `padding-box` (to inside edge of border), `content-box` (only to edge of content), `inherit`.

3.7 Color, fonts, and units

Color format	Syntax	Example
Hex code	<code>#RRGGBB</code>	<code>p{color:#FF0000;}</code>
Short hex	<code>#RGB</code>	<code>p{color:#6A7;}</code>
RGB %	<code>rgb(rr%,gg%,bb%)</code>	<code>p{color:rgb(5%,0%,0%);}</code>
RGB absolute	<code>rgb(rrr,ggg,bbb)</code>	<code>p{color:rgb(0,0,255);}</code>
HSL	<code>hsl(hue,sat%,light%)</code>	<code>hsl(0,100%,50%)</code>
Keyword	<code>name</code>	<code>p{color:teal;}</code>

- `color` is **inherited** by all descendants (e.g. setting it on `body` colors all ordinary text).
- Default `background-color` is **transparent**.
- Shorthand: `background: #fff url("cat.png") no-repeat right top;` replaces four separate `background-*` declarations.
- 5 generic font families: **Serif**, **Sans-serif**, **Monospace**, **Cursive**, **Fantasy**.
- `font-weight`: `normal`, `bold`, `bolder`, `lighter`, or 100–900.
- `font shorthand` sets `style` + `variant` + `weight` + `size` + `family` in one line.

Unit	Meaning	Example
%	Relative to another value	line-height: 125%;
em	Relative to the current font size (1em = current font size; default 16px ⇒ 1em = 16px)	font-size: 2.5em; (=40px)
px	Screen pixels	padding: 25px;
pt	Points (1pt = 1in / 72)	font-size: 18pt;
in / cm / mm / pc	Absolute physical units	margin-bottom: 2cm;

Mentor's take — Font-size in em is a common calc question: "default browser size is 16px, so pixels ÷ 16 = em." E.g. 30px → 1.875em. Practice converting both directions.

Part 4 — JavaScript

4.1 What JavaScript is

- HTML alone cannot react to users, make decisions, or automate tasks — that needs a programming/script language: **JavaScript**.
- JavaScript is the most common implementation of the standard **ECMAScript**. **JavaScript = ECMAScript**, and crucially **JavaScript ≠ Java** — different languages entirely, despite the name.
- JavaScript is **high-level** (abstracted from hardware), **dynamic** (types resolved at runtime), **untyped** (no enforced data types on variables) and **interpreted** (executed line-by-line, not pre-compiled).
- It supports both object-oriented and functional programming styles.

```
<body>
<script language="javascript" type="text/javascript">
document.write ("Hello World!")
</script>
</body>
```

4.2 Syntax basics

- JavaScript is **case-sensitive** — Name and name are different variables.
- Whitespace (spaces, tabs, newlines) is ignored — use it freely to format code.
- Comments: `//` single line; `/* ... */` multi-line.
- Variables are declared with `let` (modern) or `var` (older code). You must declare before assigning — assigning to an undeclared variable is "NOT OK."
- Statements end with a semicolon. If each statement is on its own line, semicolons are technically optional, but if two statements share a line, semicolons are **required**. Best practice: always use them.

```
let a = 1 // OK
b = 2; // NOT OK – b was never declared
let c = 3, d = 4; // OK – declares + assigns both
let e = 5; f = 6; // e is fine, f is NOT OK (never declared)
```

Scope & naming rules

- **Global** variables: defined anywhere, visible everywhere. **Local** variables: visible only inside the function where declared; function parameters are always local.
- A local variable with the same name as a global one **hides** the global inside that function.
- Variable names cannot be reserved keywords (`break`, `class`, `while`...), cannot start with a digit (must start with a letter or underscore — `_123test` is valid, `123test` is not), and are case-sensitive.

4.3 Data types and values

JavaScript types split into two categories:

- **Primitive types**: Numbers, Strings, Booleans, `null` and `undefined`.

- **Object types** (everything non-primitive): a generic **object** (named properties + values), an **array** (ordered, numbered values), and a **function** (an object with executable code attached).

JavaScript converts values between types liberally and automatically (e.g. a number is converted to a string if a string is expected). These conversion rules are not always trivial, and are a common source of bugs — see the quirks section below.

Numbers

- JavaScript makes **no distinction** between integers and floats — all numbers are floating-point.
- Operators: + - * / % (modulus), ++, --.
- + also works on strings: "a" + 10 → "a10" (the number is cast to a string).

```
Math.pow(2,53) // 2 to the power of 53
Math.round(0.6) // nearest integer
Math.ceil(0.6) // round up
Math.floor(0.6) // round down
Math.random() // random number, 0 <= x < 1
Math.sqrt(3)
Math.PI
Math.min(x,y,z) // smallest of x,y,z
Math.max(x,y,z) // largest of x,y,z
```

Strings

A string is an **immutable** sequence of characters, written in matched single or double quotes. + concatenates two strings; .length gives the character count.

```
s.charAt(n) // character at position n
s.substring(m,n) // substring from m-th to n-th char
s.indexOf(c) // first occurrence of c
s.lastIndexOf(c) // last occurrence of c
s.split(c) // split into substrings on separator c
s.replace(c1,c2) // replace every occurrence of c1 with c2
s.toUpperCase() // convert to uppercase
```

Booleans, truthy/falsy, null vs. undefined

- A Boolean has only true/false, usually the result of a comparison operator.
- **Falsy values in JS:** false, 0, -0, 0n, "", '', null, undefined, NaN, document.all, and an **empty array** ([] == false is true). Everything else is truthy (but not necessarily === true).
- undefined = a system-level / error-like absence of a value (e.g. you read a variable that was never assigned). null = a program-level / expected, intentional absence of a value.

4.4 Operators

Category	Operators
Arithmetic	+ - * / % ++ --
Comparison	== != > < >= <=
Logical	&& (AND) (OR) ! (NOT)
Bitwise logical	& (AND) (OR) ^ (XOR) ~ (unary NOT, flips all bits)
Shift	<< (left shift, ×2 per position) >> (right shift)
Assignment	= += -= *= /= %= (and bitwise versions <<=, >>=, &=, =, ^=)
Other	?: (ternary) typeof (returns "number"/"string"/"boolean" etc. as a STRING)

Example bitwise: $A=2$ (10), $B=3$ (11) $\rightarrow A \& B = 2$, $A | B = 3$, $A \wedge B = 1$, $\sim B = -4$ (32-bit signed representation, sign flip via two's-complement). Example shift: $A=2 \rightarrow A \ll 1 = 4$, $A \gg 1 = 1$.

Strict (===) vs. loose (==) equality

- `==` performs **implicit type coercion** before comparing — it converts operands to a common type first.
- `===` (strict equality) does **not** convert types — operands must match in both value AND type, otherwise they are unequal.
- This is why `true === 1` is false, but `true == 1` is true (true is coerced to the number 1 first).

The famous JavaScript quirks (these get asked verbatim — know the "why")

Expression	Result	Why
<code>[] == 0</code>	true	An empty array is coerced straight to a number $\rightarrow 0$; <code>0==0</code> is true.
<code>![] == 0</code>	true	<code>[]</code> is truthy, so <code>![]</code> is false; false is coerced to 0; <code>0==0</code> is true.
<code>typeof NaN</code>	"number"	NaN is just a number the computer cannot calculate — it is still a number.
<code>true === 1</code>	false	Strict equality never coerces types; boolean $\neq$ number.
<code>true == 1</code>	true	Loose equality coerces true $\rightarrow 1$ first.
<code>Math.max()</code>	-Infinity	-Infinity is the identity element for <code>max()</code> with zero arguments.
<code>Math.min()</code>	Infinity	Infinity is the identity element for <code>min()</code> with zero arguments.
<code>[] + []</code>	""	Both arrays convert to empty strings, then concatenate to "".
<code>{ } + []</code>	0	Leading <code>{ }</code> is read as an empty code block (ignored); <code>+[]</code> coerces <code>[]</code> to the number 0.
<code>(!+[[]+[]+![]]).length</code>	9	<code>!+[]</code> is true, <code>+[]</code> is 0 so <code>![]</code> is false; <code>true+""+false</code> concatenates to "truefalse", which has length 9.

4.5 Expressions, statements, control flow

- **Expression** — evaluated to produce a value (e.g. 10, "hello", 3+a).
- **Statement** — executed to make something happen (expression statements, conditionals, loops).

```
if (n == 5)
  m = 10;
else {
  m = 20;
  f = 3;
}
```

Loop	Shape	Key trait
while	while (expr) { ... }	Condition tested BEFORE each iteration; may run 0 times.
do / while	do { ... } while (expr);	Body runs at least ONCE before the condition is checked.
for	for (init; test; increment) { ... }	Combines initialize / test / increment in one line.
for / in	for (var c in obj) { ... }	Iterates over each property of an object.

- break exits the innermost enclosing loop/switch immediately.
- continue skips straight to the next iteration.
- switch(n) { case 1: ... break; default: ... } — remember the break on each case or execution "falls through."

4.6 Functions

```
// Function declaration
function sum(a,b) { return a+b; }

// Function expression
var sum = function(a,b) { return a+b; }

// Arrow function (ES6)
var sum = (a,b) => { return a+b; }
```

- A function is defined once but can be invoked any number of times. Code only runs when the function is INVOKED, not when it is defined.
- Functions can be assigned to variables and passed to other functions as arguments — this is fundamental for **event-driven programming** (the flow is driven by external events: clicks, key presses, etc., rather than a fixed sequence).
- Invocation styles: as a plain call `f(a)`, or as a method on an object — dot notation `o.f(a)` (common, name known ahead of time) or bracket notation `o["f"](a)` (useful when the method name is computed dynamically).

4.7 Loading JavaScript & the window/DOM model

Method	Code
External file (relative path)	<code><script src="js/myScript.js"></script></code>
External file (full URL)	<code><script src="https://example.com/js/myScript.js"></script></code>
Inline, in head/body	<code><script> ...code... </script></code>
Inline, in an event attribute	<code><button onclick="myFunction()">Click</button></code>

- Advantages of external files: separates HTML from logic, easier maintenance, a shared script is downloaded/cached once across pages, faster repeat page loads.
- Content placed between `<script src="...">` and `</script>` is simply **ignored** by the browser.

Why script order/timing matters ("why after?")

If a script tries to grab an element by ID before that element exists in the DOM yet (e.g. a script in `<head>` running before the `<body>` is parsed), `document.getElementById(...)` returns nothing useful to operate on. Two slide-given fixes:

1. Move the script to **AFTER** the target element's tag in the HTML source.
2. (More elegant) Register an event handler that waits for the whole page to load: `window.onload = function () { ... }.`

The DOM (Document Object Model)

The browser's global object when running JS is `window`. `window.document` exposes the in-memory tree representation of the page — the **DOM**. HTML elements become a tree of JS objects, each with properties and methods.

- **4 important DOM node kinds:** `document` (whole document), `element` (a tag), `attr` (an attribute), `text` (the text inside an element — an element can have several text nodes, e.g. text on either side of a nested inline tag).
- The **DOM API** lets you select nodes, traverse between nodes, manipulate them (add/replace/delete), and respond to events — directly, this is fairly verbose, which is exactly why libraries like jQuery exist.

```
// Finding elements
document.getElementById("myId1")
document.getElementsByTagName("p")
document.getElementsByClassName("myClass")
document.querySelectorAll("p.myClass")

// Changing elements
element.innerHTML = "Hello World"
element.style.color = "red";
element.setAttribute("hidden", false)

// Adding / deleting elements
document.createElement("p")
document.removeChild(document.getElementById("myId"))
document.appendChild(document.createTextNode("Hello"))
document.replaceChild(newNode, element.childNodes[0])
```

The window object's other tools

- `setTimeout()` / `setInterval()` — run a function once / repeatedly after a delay.
- `alert(msg)` — shows a message, waits for dismissal.
- `confirm(msg)` — OK/Cancel dialog, returns a boolean.
- `prompt(msg)` — asks the user for text input, returns a string.

4.8 HTML Forms in JavaScript

```
<form>
  <label for="petname">Enter your pet's name:</label><br>
  <input type="text" id="petname" name="petname" value="Daisy" required>
  <input type="submit" value="Submit">
</form>
```

- `label for="..."` must match the input's `id` to bind them.
- `name` is the key sent to the server (as a key:value pair) on submit.
- `value` sets the field's default/initial content.
- Other input types: `password` (masked), `email` (auto-validated + mobile keyboard changes), `datetime` (calendar picker), `range` (slider), `button` (runs a script function).
- `<form action="URL" method="get or post">` — `action` = submission target (defaults to current URL if omitted); `method` = which HTTP method to use.
- JS event handlers can be wired directly via attributes, e.g. `<input type="button" onclick="alert('clicked');">` — multiple statements inside the attribute are separated by semicolons.

Part 5 — jQuery, Bootstrap & AJAX

5.1 Why third-party libraries exist

Libraries streamline development, add functionality, and improve efficiency. The slides group them by role:

Category	Examples
Front-end libraries	jQuery, React
CSS frameworks	Bootstrap
Back-end libraries	Django, Flask
Database libraries	Sequelize
Testing libraries	Jest
Template engines	Jinja2

5.2 jQuery

jQuery is an open-source library written entirely in JavaScript that **simplifies DOM API interaction** (and other tasks like AJAX), provides an easy-to-learn API, and gives **cross-browser support** — consistent behavior on Chrome, Firefox, Safari, Edge, etc. It became the de-facto standard for web development.

- Add to a page by (1) downloading the file and self-hosting it, (2) loading from a CDN (e.g. `code.jquery.com` — with an integrity hash + `crossorigin` attribute for tamper-checking), or (3) loading from Google's CDN (often already cached in the visitor's browser → faster load).
- Core export: a single function aliased as `$`. Basic syntax: `$(selector).action()`.

```
$("#td"); // all <td> elements
$("#contacts"); // element with id="contacts"
$(".controls"); // elements with class="controls"
$("body h1"); // all <h1> inside <body>

$("p").hide(); // hide all <p>
$("h1").text(); // get concatenated text of all <h1>
$("h1").text("hello"); // set text of all <h1>
```

`$("#myElement").hide();` is shorter/more readable than the raw DOM API equivalent `document.getElementById("myElement").style.display = "none";` — this is jQuery's whole pitch.

DOM manipulation, styling, and the "this" gotcha

Function	Effect
.before() / .after()	Insert content adjacent to the element (does NOT touch its children).
.append() / .prepend()	Insert content AS a child (last / first), of the selected element.
.text() / .html()	Get/set the text or HTML content of an element.
.addClass() / .removeClass()	Add/remove a CSS class.
.css("prop","value")	Set a CSS property directly (whichever rule applies LAST, by specificity + order, wins — a jQuery .css() call after page load typically overrides an earlier CSS rule of equal specificity).

- Event callbacks: `.click(fn)`, `.mousemove(fn)`, `.change(fn)`, `.hover(fn)`, `$(document).ready(fn)`.
- Inside a normal function passed to a handler, `this` refers to the element that generated the event: `$(this).text("Clicked!")`.
- **Gotcha:** arrow functions `() => {}` do NOT have their own `this` — they inherit it from the surrounding lexical scope, which is rarely what you want in a handler. Use event `.target` instead when using arrow functions.

```
$("#myButton").on("click", (event) => {
  console.log(event.target); // the clicked element – NOT `this`
});
```

- Animations: `.show()`, `.hide()`, `.toggle()`, `.fadeIn()`, `.fadeOut()`, `.slideUp()`, `.slideDown()`.

5.3 Bootstrap

Bootstrap is a free, open-source **CSS framework** for responsive, mobile-first front-end development, bundling HTML/CSS/(optional JS) templates for typography, forms, buttons, navigation, etc. **Bootstrap 5**, released in **2021**, is the current major version and **dropped the jQuery dependency** that Bootstrap 3/4 had, to align with modern JS practice.

Responsive Web Design (RWD) principles

- Fluid grid — size elements with relative units (percentages), not fixed pixels.
- Images sized in relative units too.
- **Media queries** apply different CSS based on the device's viewport (width, height, orientation, resolution); breakpoints define where the styling changes.

	Adaptive design	Responsive design
Approach	Multiple FIXED layouts, one per device class, served via server-side device detection.	ONE fluid layout that reflows across all screens via relative grids + media queries.
Codebase	Potentially several tailored versions.	Single codebase for every device.

The grid system

- Built on **containers** → **rows** → **columns**. `.container` = fixed max-width per breakpoint; `.container-fluid` = always 100% viewport width.
- Up to **12 columns** per row; you can merge adjacent column classes to make wider columns.
- Content must sit inside columns; only columns may be direct children of a row.

Breakpoint	Min. viewport width	Container max-width	Class prefix
Extra small	< 576px	100% (auto)	.col-*
Small	≥ 576px	540px	.col-sm-*
Medium	≥ 768px	720px	.col-md-*
Large	≥ 992px	960px	.col-lg-*
Extra large	≥ 1200px	1140px	.col-xl-*
XXL	≥ 1400px	1320px	(.col-xxl-* in BS5)

```
<div class="container-md border border-danger">
  <div class="row">
    <div class="col-md-3 col-sm-6">col 1</div>
    <div class="col-md-3 col-sm-6">col 2</div>
    <div class="col-md-3 col-sm-6">col 3</div>
    <div class="col-md-3 col-sm-6">col 4</div>
  </div>
</div>
```

Reading the example above: on **medium** screens, the 4 columns span equal space in **one row** (3+3+3+3=12). On **small** screens they pair up and **stack into 2 rows** (6+6=12 each). On **extra-small** screens (no matching class) they each take the full width and stack into **4 rows**.

5.4 AJAX & asynchronous programming

Asynchronous programming lets a program run tasks **concurrently** without blocking the main execution flow — the program keeps responding while a slow operation (network call, file I/O, DB query) is still in progress. Main benefits: better performance and responsiveness.

AJAX (Asynchronous JavaScript And XML) is a set of browser techniques that let a page exchange data with a server **in the background, without reloading** the page. Historically used XML; **modern implementations use JSON** for its simplicity and native fit with JS objects.

The AJAX request/response cycle

1. An event occurs on the page (load, click, etc.).
2. JavaScript creates an XMLHttpRequest object.
3. That object sends a request to the server.
4. The server processes the request.
5. The server sends a response back.
6. JavaScript reads the response.
7. JavaScript performs the proper action (e.g. updates the page) without a full reload.

Single Page Applications (SPA)

- Dynamically rewrite the current page instead of loading whole new pages from the server.
- Typically a single HTML "shell" file; content loads asynchronously.
- Heavy reliance on JavaScript for rendering, input handling, and server communication.
- Examples given in the slides: Gmail, Google Maps, Twitter, Facebook.

JSON (JavaScript Object Notation)

- A lightweight data-exchange format, easy for humans to read and for JS to parse directly into objects.
- **Object/Dictionary**: { }, name:value pairs separated by commas.
- **Array**: [], ordered values separated by commas.
- **Value** can be a string (in double quotes), number, true/false, null, an object, or an array — and these can nest.

```
{
  "books": [
    {"title": "1984", "author": "George Orwell",
     "year_published": 1949, "genre": "Dystopian"}
  ]
}
```

AJAX with jQuery

```
// Shorthand for a GET request expecting JSON
$.getJSON(url, data, function(d) {
  // d = parsed JS object from the JSON response
});

// The general-purpose form
$.ajax({
  type: "GET", // or "POST"
  url: url,
  data: null,
  dataType: "json",
  success: function(response) { /* handle it */ }
});
```

Mentor's take — A very testable distinction: **server-side** (server owns app logic, input validation, and HTML generation; client just renders) vs. **client-side/SPA** (the client owns presentation AND app logic; server exchanges only raw data). Asynchronous programming + AJAX is the technology that made the client-side model practical.

Part 6 — HTTP, Web Servers & Flask

6.1 HTTP, the protocol underneath everything

Per **RFC 2616**: HTTP is "an application-level protocol for distributed, collaborative, hypermedia information systems... a generic, stateless protocol." Key properties, all of which are individually testable:

- **Request-response**: a client sends a request message; the server returns a response message — asymmetric (client and server play different roles).
- **Pull protocol**: the client pulls information from the server (the server never pushes unprompted).
- **Stateless**: the current request has no knowledge of previous requests.

A typical transaction: browser turns a URL into a request message → sends it to the server → server interprets it → server returns a response (the resource, or an error).

6.2 URL anatomy (the 4-part practical view used for HTTP)

`protocol://hostname:port/path-and-file-name`

Part	Meaning	Example (http://www.example.com/docs/index.html)
Protocol	Application-level protocol (HTTP, FTP, telnet...)	HTTP
Hostname	DNS domain name or IP of the server	www.example.com
Port	TCP port the server listens on	Not specified → defaults to 80 for HTTP
Path-and-file-name	Resource location under the server's document root	/docs/index.html

6.3 Request line, status line, headers

Request line: request-method-name request-URI HTTP-version

e.g. GET /test.html HTTP/1.1
 HEAD /query.html HTTP/1.0
 POST /index.html HTTP/1.1

Status line: HTTP-version status-code reason-phrase

e.g. HTTP/1.1 200 OK
 HTTP/1.0 404 Not Found
 HTTP/1.1 403 Forbidden

Both request and response headers are name: value pairs. A message body (payload) is optional and is separated from the headers by a blank line.

6.4 HTTP methods — the most exam-relevant table in this whole course

Method	Purpose	Safe?	Idempotent?
GET	Read-only retrieval of a resource.	Yes	Yes — repeated identical GETs return the same result.
POST	Create / add a new item (e.g. new account, new blog post). Success → status 201 Created.	No	No — repeating it duplicates the resource.
PUT	Modify or fully replace existing data at a known URL (e.g. change a password). Can also create.	No	Yes — repeating it updates the same resource to the same end state.
DELETE	Delete the targeted resource. Risky — once deleted it cannot come back.	No	Yes (per slides) — though a 2nd identical request often returns 404 since the resource is already gone.
HEAD	Like GET but returns only headers (metadata: size, last-modified), no body.	Yes	Yes
OPTIONS	Ask the server which methods are supported for a resource.	Yes	Yes
PATCH	Apply a PARTIAL modification to a resource (vs. PUT's full replace).	No	No (generally)
CONNECT	Establish a tunnel to a server, e.g. for HTTPS/proxying.	No	N/A
TRACE	Echoes back the exact request received — diagnostic tool.	Yes	Yes

- **Safe** = the method has NO effect on server state — effectively read-only.
- **Idempotent** = making the same request repeatedly produces the same end result.
- **Cacheable** = responses MAY be stored for reuse (GET/HEAD typically yes; POST/PUT/DELETE typically no).
- PUT vs. POST, the classic distinction: **POST creates, is not idempotent, typically used in HTML forms.**
PUT updates/creates a specific resource, is idempotent, typically used in programmatic APIs.

Mentor's take — The sample exam asks "what is the primary purpose of the POST method?" — answer: "to create or add a new item," NOT "to update a resource" (that's PUT/PATCH) and NOT "to retrieve data" (that's GET). The wording trap is mixing up POST with PUT — don't fall for it.

- Ways a browser issues requests with specific methods: typing into the **address bar** (always a GET), an HTML `<form action="URL" method="get/post">`, or jQuery/AJAX calls.

6.5 Web servers and frameworks

A **web server** is a computer system that delivers web content (HTML, images, CSS, JS) to end users over the Internet via a browser, using HTTP request/response.

Framework	Language	Famous use cases
Django	Python	Instagram, Pinterest, Coursera
Flask	Python	Red Hat, Rackspace, Reddit
ExpressJS	Node.js	MySpace, GeekList, Storify
Laravel	PHP	Deltanet Travel, Neighborhood Lender
Ruby on Rails	Ruby	Zendesk, Shopify, GitHub
ASP.NET	C#	Microsoft, GoDaddy, Ancestry
Spring Boot	Java	Trivago, Via Varejo, Intuit
Phoenix	Elixir	Financial Times, Fox 10, ABC15

6.6 Flask essentials — this is your project's backbone

Flask is a Python **microframework**: ready to go, lightweight, flexible — but with fewer built-in tools, so complexity grows as the project grows. Requires Python ≥ 3.7 and Flask ≥ 2.2 .

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def hello_world():
    return "<html><body><p>Hello World!</p></body></html>"
```

- By default a Flask app listens on `127.0.0.1:5000` (localhost only).
- Run from the terminal: `flask --debug --app myapp.py run` (or `python3 -m flask --debug --app myapp.py run`).
- The `@app.route()` decorator maps a URL to the function below it; the function's return value is the HTTP response body.

Routing with variables and types

```
@app.route("/<name>")
def hello_you(name):
    return "<p>Hello, {}</p>".format(name)

@app.route("/<int:age>")
def hello_you2(age):
    return "<p>Hello, I am {} years old.</p>".format(age)
```

Variable sections use `<variable_name>`; the view function receives it as a keyword argument. You can constrain the type with `<type:name>` — supported types include `string`, `int`, `float`, `path`, `uuid`.

Static files

Flask automatically serves anything inside a `static/` subfolder. You can also serve files programmatically with `send_from_directory()` — useful for a download route.

```
from flask import Flask, send_from_directory
app = Flask(__name__)

@app.route('/<filename>')
def download_file(filename):
    return send_from_directory('static', filename, as_attachment=True)
```

Templates & Jinja2

Generating HTML from raw Python strings does not scale, so Flask uses the **Jinja2** template engine: write near-Python syntax inside HTML files, then render with data.

```
from flask import Flask, render_template
app = Flask(__name__)

@app.route('/hello/<value>')
def hello(value):
    return render_template('home.html', name=value)
```

```
<!-- templates/home.html -->
<h1>Hello {{ name }}!</h1>
```

Jinja2 delimiter	Used for
{% ... %}	Statements (if, for, extends, block...)
{{ ... }}	Expressions printed to the output
{# ... #}	Comments — never appear in the rendered output
# ... ##	Line statements (a one-line shorthand for {% %})

Template **inheritance** lets a child template reuse a shared layout: {% extends "base.html" %} plus a matching {% block content %} ... {% endblock %} in both files.

The request object

Flask exposes a global request object so a view can react to data the client sent (form fields, query parameters, the HTTP method used, cookies).

```
from flask import Flask, request, render_template
app = Flask(__name__)

@app.route('/', methods=['POST', 'GET'])
def main():
    if request.method == 'POST':
        return login()
    return render_template("login.html")

def login():
    username = request.form.get('username')
    password = request.form.get('password')
    if valid_login(username, password):
        return render_template('welcome.html', name=username)
    return render_template('login.html', error='Invalid username/password')
```

Mentor's take — Notice `methods=[ 'POST', 'GET' ]` on the route — by default a route only accepts GET. Forgetting to whitelist POST on a login route is a real bug, and recognizing that pattern is exactly the kind of thing an open question or a project review will probe.

Part 7 — Sessions, Cookies & Authentication

7.1 From stateless to stateful

HTTP is anonymous and stateless by design — every request, even from the same client, is independent of earlier ones. But real applications need to recognize a returning client and attach data to it (login state, shopping cart, preferences). That application data tied to a specific client is called a **session**. The key idea: **HTTP is stateless, but not "sessionless" — sessions are built on top of it**, most commonly using cookies.

7.2 What a cookie is and what it is for

A **cookie** (web/browser cookie) is a small piece of data a server sends to a browser; the browser stores it and sends it back to the same server on later requests. Cookies are simple key-value pairs.

Purpose	Example
Session management	Sign-in status, shopping-cart contents, game scores.
Personalization	Display language, UI theme/preferences.
Tracking	Recording and analyzing user behavior.

A simple sign-in flow using cookies

1. User sends sign-in credentials to the server.
2. If correct, the server responds with a cookie containing a session ID that records the sign-in status.
3. Later, the user navigates to another page on the same site; the browser automatically attaches that session-ID cookie to the new request.
4. The server checks the session ID: if still valid, it returns a personalized page; if invalid, the session is deleted and the user sees a generic page / "access denied" / sign-in page.

Cookies travel inside **HTTP headers**, so they are invisible to the user directly: the server issues Set-Cookie headers; the browser echoes stored cookies back in a Cookie header on every subsequent request to that domain.

```
HTTP/2.0 200 OK
Set-Cookie: yummy_cookie=choco
Set-Cookie: tasty_cookie=strawberry

GET /sample_page.html HTTP/2.0
Cookie: yummy_cookie=choco; tasty_cookie=strawberry
```

7.3 Cookie types and the attributes that control them

Type	Lifespan
Session cookie (transient)	Lives only as long as the browser session; deleted when the browser closes.
Persistent cookie	Lives until an explicit expiry; survives a browser/PC restart.

```
Set-Cookie: name = value [; expires=date] [; path=path] [; domain=domain] [; secure] [; HttpOnly]
[; SameSite]
```

Attribute	Effect
expires / Max-Age	Sets an explicit expiry date / lifetime in seconds. Absent ⇒ session cookie (deleted when "the current session" ends, as defined by the browser).
path	Restricts the cookie to a resource (and its sub-paths), e.g. path=/app sends the cookie only to /app and its subdirectories.
domain	Restricts the cookie to a domain + its subdomains.
secure	Browser sends the cookie ONLY over HTTPS (secure connections).
HttpOnly	Blocks JavaScript from reading the cookie (e.g. via document.cookie) — mitigates XSS theft.
SameSite	Controls whether the cookie is sent with cross-site requests — helps prevent CSRF.

7.4 Security vs. privacy — two different concerns

Concept	Goal
Security	Protect systems/networks/data from unauthorized access, attacks, damage — integrity, availability, confidentiality (encryption, authentication, authorization, audits, firewalls...).
Privacy	Proper handling of personal data and individual rights over how it is collected, used, shared — user control, data minimization, consent, compliance with regulations.

- Why cookie risk is underestimated: (1) cookies CAN be disabled, so people treat them as "the user's responsibility"; (2) there is a false sense that "data left on my own browser can't be harmful."
- **Third-party cookies** (set by embedded third-party content, e.g. an ad banner) enable invasive tracking — the classic "ads for a product follow you around the web" experience.
- Regulations with global reach because they apply to any site accessed from those jurisdictions: **GDPR** (EU), the **ePrivacy Directive** (EU), the **California Consumer Privacy Act**. Common requirements: notify users of cookie use, allow opt-out of some/all cookies, allow most of the service to work without cookies.

7.5 Authentication

Cookies recognize a returning client, but not its real identity. **Authentication** means proving the true identity of a client, usually via shared secret information (e.g. username/password) exchanged through a defined protocol. The general flow is the same across schemes — server challenges, client responds — but the encoding differs.

Basic authentication (HTTP/1.0)

1. Server challenges the client for a username:password pair.
2. User supplies it.
3. The pair is encoded with **Base64**.
4. The encoded pair is sent in an HTTP header.

Base64 is encoding, NOT encryption — it is trivially reversible by anyone who intercepts it. Basic auth therefore offers the **same (very poor) security level as sending the username/password in plain text** unless the connection itself is secured (HTTPS).

Digest authentication

Rationale: the server does not strictly need the password itself — only proof the client knows it. So the client sends a **digest** (a hash), never the password in clear text.

- A good cryptographic hash function must be: quick to compute, NOT invertible (only brute force can find a message matching a given digest), tamper-resistant (any change to the message changes the digest), and collision-resistant (practically impossible to find two different inputs with the same digest).
- **Problem:** the same password always produces the same digest, so an eavesdropper who captures the digest could simply **replay** it later to authenticate as the user — a **replay attack**.
- **Fix:** the server supplies a unique single-use **nonce** ("number used only once"); the client hashes (password, nonce) together, so the digest is different every time and a captured one becomes useless once the nonce expires.

7.6 HTTPS = HTTP over SSL/TLS

Plain HTTP request/response bodies are **plaintext** — trivially readable/modifiable by an attacker on the network. **HTTPS** inserts a security layer (**SSL/TLS** — Secure Sockets Layer / Transport Layer Security) between HTTP (application layer) and TCP (transport layer) to encrypt the conversation.

- **Asymmetric cryptography:** a certificate has a private part (kept secret, used to prove your own identity) and a public part (handed to others for verification).
- **SSL handshake:** the browser receives the server's digital certificate and validates it (warns the user if untrusted); it extracts the server's public key, generates a random secret key, encrypts that secret with the public key, and sends it back — this is how client and server agree on a shared secret key for the session.
- **Certificate Authorities (CAs)** issue certificates; trusting a certificate means trusting the CA that issued it.

7.7 Cookie attack categories (named threats — good MC/open material)

Threat	How it works	Mitigation
Network threats	Cookies sent over an unencrypted channel can be eavesdropped.	Use SSL/TLS (HTTPS); encrypt only the sensitive payload inside the cookie if full HTTPS is not possible.
End-system threats	Cookies stored in clear text on disk/memory can be stolen, copied to another machine, or altered to impersonate the user; reading them from a browser/proxy cache is called "cache sniffing."	Avoid storing sensitive data directly in cookies; sign/encrypt cookie values; short expirations.
Cookie harvesting	An attacker impersonates a legitimate site to collect cookies, or embeds cross-site scripting (XSS) in a link (forum post, email) that activates when the victim clicks it.	HttpOnly to block script access; CSP; input sanitization; user education.

The Secure attribute (RFC 2965, optional) limits a cookie's scope to "secure" channels — typically HTTPS — meaning the browser will only attach it to requests sent over an encrypted connection.

7.8 Cookies in Flask (practical syntax for your project)

```
resp = make_response(render_template('readcookie.html'))
resp.set_cookie('userID', user) # set
return resp

name = request.cookies.get('userID') # read
```

`Response.set_cookie(key, value='', max_age=None, expires=None, path='/', domain=None, secure=None)` — `max_age` defaults to the client's browser session; `secure=True` sends it only over HTTPS.

Mentor's take — Your project topic involves a booking flow with logged-in users and personalized recommendations — that is a session-management problem end to end. Plan now whether you will roll your own cookie/session logic or use Flask's built-in session object (which signs data server-side); either is defensible, but say which one and why in your write-up.

Part 8 — Databases & SQL

8.1 Why a DBMS instead of a plain file

You could store structured data in something simple like a CSV file. A **Database Management System (DBMS)** becomes necessary once you have: data of different kinds (binary, text...), complex relationships between items, possibly large datasets, and **concurrent** read/write access from multiple users.

Common database models (know the trade-off in one line each)

Model	One-line trade-off
Flat	Single table — simple, but cannot represent complex relationships well.
Hierarchical	Tree of parent-child records — efficient for hierarchies, bad for many-to-many.
Relational	Tables + rows/columns + keys; SQL query language; normalization keeps data integrity.
Star schema	One central fact table + dimension tables — optimized for fast analytical queries.
Snowflake	Star schema with normalized (split) dimension tables — less redundancy, more joins.
Network	Each record can have multiple parents/children (a graph) — flexible many-to-many.

8.2 Concrete vs. abstract knowledge (the conceptual modeling step)

Modeling a real system into a database means organizing two kinds of knowledge — using a "students and exams" example throughout the slides:

Knowledge type	Definition	Example
Concrete	Entities, properties, relationships.	Entity: student, exam. Property: student name/ID/email; exam date. Relationship: a student has done many exams.
Abstract	Formal restrictions on what values concrete knowledge can take.	Name is a string ≤ 256 chars; each student has ONE study plan (unary); a study plan refers to MANY exams (n-ary).

- Concrete/abstract knowledge describe the **organization** of data and should change rarely — it is hard to change a model once the DB is in use, and keeping a wrong model makes the DB inefficient. **The data itself** (actual rows) changes constantly, but the schema should not.

8.3 From knowledge to tables

- A **table** represents an entity; a **row** is one instance of that entity; a **column** holds one property.
- A **key** is a minimal subset of columns that uniquely identifies a row (sometimes more than one column is needed).
- If more than one key exists (candidate keys), the designer picks one as the **primary key**. The DBMS typically auto-indexes the primary key, making lookups by it faster.
- A **foreign key** is a key from one table used in another table to implement a relationship — its value must correspond to an existing primary/unique key value in the referenced table.


```
-- students
name surname student_ID birthdate email
Albert Einstein 0001 1879-03-14 albert@gmail.com

-- exam
course_title course_ID date student_ID grade
Software Development ET7018 2018-12-18 0001 31
```

In the exam table, student_ID is a foreign key pointing back to students. The natural primary key for the exam table is the **composite** (course_ID, student_ID).

8.4 Database and DBMS definitions

- A **database** = a collection of **persistent** data split into the **schema** (organization rules — structure + validity constraints, stable over time) and the **data** (time-variant facts).
- Properties of data in a DB: persistent (exists until explicitly deleted); accessed through **atomic transactions** (a transaction saves ALL its changes or NONE); protected from unauthorized access and from hardware/software failure; shareable concurrently by multiple users.
- A **DBMS** is software that lets you define a schema, choose storage strategy, and access/retrieve data via queries/programs. Administrators define the schema; the DBMS also has a physical data representation the admin usually does not need to manage directly; many DBMSs support access control (users with specific privileges).

8.5 SQL: a declarative language with 4 sublanguages

Sublanguage	Acronym	Purpose
Data Definition Language	DDL	Define the schema: CREATE TABLE, ALTER TABLE, DROP TABLE.
Data Query Language	DQL	Retrieve data: SELECT.
Data Manipulation Language	DML	Modify data: INSERT, UPDATE, DELETE.
Data Control Language	DCL	Manage access/permissions.

SQL is **declarative**, not procedural: you describe WHAT data you want and the DBMS figures out how to get it, instead of writing a step-by-step procedure (as you would in Python, a procedural language). This is why applications still use a procedural language (Python) for the app logic, calling into SQL only for the data layer — though SQL does include some procedural extensions for convenience.

DDL — defining tables

```
CREATE TABLE student (
  name CHAR(256) NOT NULL,
  surname CHAR(256) NOT NULL,
  student_ID CHAR(8) NOT NULL,
  birthdate CHAR(10) NOT NULL,
  email CHAR(256) NOT NULL,
  PRIMARY KEY (student_ID),
  CHECK (email LIKE ('%@%'))
);
```

- No procedures — you just declare the shape you want. Every column has a name, type, and size.
- Constraints: NOT NULL (must not be empty), CHECK(...) (custom validity rule, e.g. the email must contain "@").
- By convention SQL keywords are written in ALL CAPS (not mandatory). Statements always end in ;. A primary key can span multiple columns.
- DROP TABLE exam removes a table; ALTER TABLE exam ADD COLUMN exam_type CHAR(10) NOT NULL; modifies one (ALTER also supports DROP and MODIFY).

DML — inserting data

```
INSERT INTO student (name, surname, student_ID, birthdate, email)
VALUES ('Albert', 'Einstein', '0001', '1879-03-14', 'albert@gmail.com');
```

DQL — querying data

```
SELECT name, surname FROM student; -- choose columns
SELECT * FROM student; -- all columns
SELECT * FROM exam WHERE grade > 30; -- filter rows
SELECT name FROM student WHERE student_ID='0001' OR student_ID='0002';
```

JOIN — and why the naive version is wrong

JOIN alone produces the **Cartesian product** of two tables — every row of A combined with every row of B (3 rows × 3 rows = 9 rows). That includes **mismatched** combinations. You almost always need a WHERE clause matching the foreign key to the primary key it references:

```
-- Wrong (or at least incomplete): Cartesian product, mismatches included
SELECT * FROM student JOIN exam;

-- Right: keep only rows where the keys actually match
SELECT student.name, student.surname FROM student JOIN exam
WHERE student.student_ID = exam.student_ID
  AND exam.grade = 30;
```

Mentor's take — This exact "JOIN gives you a Cartesian product, then WHERE narrows it to matching rows" point is one of the most concrete, testable facts in the whole DB lecture. If an open question asks you to explain JOIN, lead with the Cartesian product, then the matching condition.

When you JOIN, you must qualify repeated column names with the table name (student.name instead of just name) — mandatory only when the name is ambiguous, but it is good practice regardless.

8.6 SQLite, prepared statements, and password hashing — the Python side

- **SQLite** is a "serverless" SQL engine — no TCP socket, no separate server process; it is just a local file accessed like opening/closing a regular file. Good for local configs / small projects; large concurrent multi-host systems need a real client-server DBMS instead. The SQL you write is the same regardless.
- **Prepared statements** use ? placeholders so the query text and the variable values are sent to the DBMS **separately** — this is the standard defense against SQL injection.

```
rows = conn.execute(
    "SELECT * FROM user WHERE username=? AND password=?",
    (username, digest)
)
```

- A **hash function** converts an input of any length into a fixed-length, irreversible string. Password databases store the **hash** of a password, never the password itself; on login the entered password is re-hashed and compared.
- A **salt** (a random value, stored alongside the hash) is concatenated with the password before hashing, so the same password never produces the same stored hash twice — this defeats precomputed "rainbow table" attacks and stops two users with the same password from having identical hashes.

```
import hashlib, random
salt = str(random.random())
digest = hashlib.sha256((salt + password).encode('utf-8')).hexdigest()
```

8.7 The "Cookbook App" exercise (context for hands-on practice)

Lecture 15's practical exercise builds a small Flask + SQLite app with user registration/login/logout, recipe creation (dish name/type/description/dietary flags), and search by dish type while excluding undesired ingredients. It explicitly demonstrates: hashed passwords (`werkzeug.security`), protected routes for logged-in users only, two related tables (users, recipes), and Jinja2 template inheritance via a shared `layout.html`. This is essentially a smaller version of the booking-app architecture your own project needs.

Part 9 — Quick-Reference Cheat Sheets

One last honest note before you cram these: this page is a compressed index, not a substitute for reading Parts 1–8. Use it the night before to test recall — if a row does not immediately make sense, go back to the matching Part and re-read it properly.

CS-1 — CSS Specificity Calculator

Selector type	Points
Inline style attribute	1000
ID (#id)	100
Class / pseudo-class / attribute ([attr], .cls, :hover)	10
Element / pseudo-element (div, ::before)	1
Universal selector (*), combinators (>, +, ~, space)	0

CS-2 — HTTP Methods at a Glance

Method	Safe	Idempotent	Typical use
GET	Yes	Yes	Read a resource
HEAD	Yes	Yes	Read headers only
POST	No	No	Create a resource
PUT	No	Yes	Replace/update a resource
PATCH	No	No	Partially update a resource
DELETE	No	Yes	Remove a resource
OPTIONS	Yes	Yes	List supported methods

CS-3 — Cookie Attributes

Attribute	What it controls
expires / Max-Age	When the cookie dies (absent → session cookie).
path / domain	Which URLs / which (sub)domains receive the cookie.
secure	HTTPS-only transmission.
HttpOnly	Blocks JavaScript access (anti-XSS).
SameSite	Cross-site sending rules (anti-CSRF).

CS-4 — JavaScript Falsy Values (memorize the full list)

false, 0, -0, 0n, "", '', null, undefined, NaN, document.all, and an **empty array** when loosely compared ([] == false). Everything else is truthy.

CS-5 — Block vs. Inline HTML Elements

Block	Inline
div, h1–h6, p, ol, ul, li, nav, form, table	span, a, img, em, b, i, br, small, button

CS-6 — TCP/IP Five Layers, Top to Bottom

Application → Transport → Network → Data Link → Physical. (Mnemonic: "**All Top Networks Deserve Praise.**")

CS-7 — CSS Selector Combinators

Pattern	Means
A B	B is a descendant of A, any depth.
A > B	B is a DIRECT child of A.
A, B	Selects A and B (a grouped list, not a relationship).
A.cls	Element A with class cls.
A:hover	Element A while the mouse is over it.

CS-8 — Flask Quickstart Skeleton

```
from flask import Flask, render_template, request
app = Flask(__name__)

@app.route('/', methods=['GET', 'POST'])
def index():
    if request.method == 'POST':
        ...
    return render_template('index.html')

if __name__ == '__main__':
    app.run(debug=True)
```

CS-9 — SQL in Four Lines

```
CREATE TABLE t (id INTEGER PRIMARY KEY, name CHAR(50) NOT NULL);
INSERT INTO t (id, name) VALUES (1, 'Ada');
SELECT * FROM t WHERE id = 1;
SELECT a.*, b.* FROM a JOIN b WHERE a.id = b.a_id; -- never forget the WHERE
```

Part 10 — Mock Exam (Expanded Practice Set)

The official sample exam on Moodle ("Lab of Web Technologies Sample Final Exam," 2024-12-17) follows the real exam format exactly: **5 multiple-choice questions** (one correct option out of five) plus **3 open-ended questions**. The official exam itself follows that same 5+3 structure.

This practice set below contains **all 8 original questions verbatim**, plus **5 additional multiple-choice** and **3 additional open-ended** questions written in the same style and pulled from across the whole syllabus — doubling the practice volume to **10 multiple-choice + 6 open-ended questions**. Every question has its correct answer (and, for the open questions, a full model answer) directly underneath it, exam-style.

Mentor's take — Cover the answer boxes with your hand/a sheet of paper and answer for real before you check. Reading a model answer passively feels like studying but barely moves the needle — retrieving the answer yourself, then checking, is what actually sticks. This is not a tip, it is the single highest-leverage thing you can do with the next hour before the exam.

Section A — Multiple Choice (Questions 1–10)

Questions 1–5 are reproduced exactly from the official sample exam. Questions 6–10 are new, covering additional topics from the syllabus.

Question 1. What does Web 3.0 aim to achieve?

- A. Development of static web pages
- B. Making Internet data machine-readable
- C. Enhancing the HTML language
- D. Increasing web browser competition
- E. Reinventing file transfer protocols

Correct answer: Option **B** — Making Internet data machine-readable (*Web 3.0 / the Semantic Web aims to make Internet data machine-readable so software agents can analyze content, links, and transactions directly.*)

Question 2. What is the purpose of CSS?

- A. To add interactivity to web pages
- B. To define the structure of documents
- C. To separate content from presentation
- D. To execute tasks concurrently
- E. To improve user experience by loading data asynchronously

Correct answer: Option **C** — To separate content from presentation (*Structure is HTML's job and interactivity/async loading is JavaScript/AJAX's job; CSS's defining purpose is separating presentation from content.*)

Question 3. How is an external JavaScript file loaded into an HTML document?

- A. `<script>myScript.js</script>`
- B. `<script src="myScript.js"></script>`
- C. `<link rel="script" href="myScript.js">`
- D. `<javascript src="myScript.js"></javascript>`
- E. `<import src="myScript.js"></import>`

Correct answer: Option **B** — `<script src="myScript.js"></script>` (The `src` attribute on a `<script>` tag is the standard way to load an external JS file; content between the tags is ignored when `src` is present.)

Question 4. What is the primary purpose of the POST method in HTTP?

- A. To retrieve data
- B. To delete data
- C. To create or add a new item
- D. To update a resource
- E. To read the server response

Correct answer: Option **C** — To create or add a new item (*POST creates/adds a new item (e.g. a new account or blog post) and is not idempotent — repeating it duplicates the resource. Updating an existing resource is PUT/PATCH's job.*)

Question 5. What is the purpose of SSL and TLS?

- A. To block cookies
- B. To provide secure communication
- C. To speed up data transmission
- D. To compress web pages
- E. To enable user tracking

Correct answer: Option **B** — To provide secure communication (*SSL/TLS sit between HTTP and TCP to encrypt the conversation, turning HTTP into HTTPS.*)

Question 6. Which of the following best describes the difference between the Internet and the World Wide Web?

- A. They are different names for the exact same technology
- B. The Internet is a system of hypertext documents, while the WWW is the physical cabling and router infrastructure
- C. The Internet is the global network of interconnected computers using TCP/IP; the WWW is a system of interlinked hypertext documents accessed through that network
- D. The World Wide Web existed before the Internet was invented
- E. The Internet only supports email, while the WWW supports browsing

Correct answer: Option **C** — The Internet is the global network of interconnected computers using TCP/IP; the WWW is a system of interlinked hypertext documents accessed through that network (*The Internet is the underlying network (TCP/IP); the Web is one application built on top of it, using HTTP/HTTPS and HTML to link documents.*)

Question 7. According to CSS specificity rules, how many points does a single ID selector (e.g. #navbar) contribute to a rule's specificity?

- A. 1
- B. 10
- C. 100
- D. 1000
- E. 0 — IDs carry no specificity

Correct answer: Option **C** — 100 (*The specificity formula adds 1 per element, 10 per class/pseudo-class/attribute selector, 100 per ID, and 1000 for an inline style attribute.*)

Question 8. Which HTTP method is both "safe" (has no effect on server state) AND idempotent (repeating it produces the same result)?

- A. POST
- B. PUT
- C. DELETE
- D. GET
- E. PATCH

Correct answer: Option **D** — GET (*GET is read-only (safe) and returns the same result every time it is repeated (idempotent). PUT/DELETE are idempotent but not safe (they change server state); POST/PATCH are neither.*)

Question 9. In a relational database, what is the purpose of a foreign key?

- A. To uniquely identify a row within its own table
- B. To encrypt sensitive values stored in a column
- C. To identify a row in a different table, implementing a relationship between two entities
- D. To set the maximum number of rows a table may contain
- E. To automatically generate new primary key values

Correct answer: Option C — To identify a row in a different table, implementing a relationship between two entities (*A foreign key is a key that points to a row in another table; that is precisely how relational databases implement relationships between entities.*)

Question 10. What is the key difference between the == and === operators in JavaScript?

- A. == checks object reference equality, while === checks only value equality
- B. === performs implicit type coercion before comparing, while == does not
- C. == performs implicit type coercion before comparing values, while === (strict equality) requires both value and type to match without converting either operand
- D. There is no real difference; the two operators are fully interchangeable
- E. === can only be used to compare strings, never numbers or booleans

Correct answer: Option C — == performs implicit type coercion before comparing values, while === (strict equality) requires both value and type to match without converting either operand (*This is exactly why true === 1 is false (different types, no coercion) while true == 1 is true (true is coerced to the number 1 before comparing).*)

Section B — Open-Ended Questions (Questions 11–16)

Questions 11–13 are reproduced exactly from the official sample exam (there, numbered 6–8). Questions 14–16 are new. Model answers below are written to the level of detail an exam grader would expect, based directly on the course slides.

Question 11. In what ways does separating structure from presentation facilitate better web design and user experience?

Model / correct answer: Three reasons, straight from the lecture on HTML: **(1) Division of labor** — authors/programmers and visual designers have different skills; separating structure (HTML) from presentation (CSS) lets authors focus on content while designers control appearance, and designers can enforce one consistent style across an entire site or organization. **(2) Fast, low-risk changes** — if markup contains only content and structure, the document's appearance can be changed quickly without touching the content: different presentations can be defined for different media (screen vs. print vs. aural devices), and an entire website's look can be updated by editing a shared stylesheet rather than every page. **(3) Accessibility** — the same structured document can be rendered very differently depending on the user's needs, e.g. read aloud by a screen/text reader for users with limited hearing, or output to a Braille printer for users with limited vision, because the presentation layer is not hard-coded into the content.

Question 12. What is asynchronous programming? Discuss the role of AJAX in creating dynamic web applications.

Model / correct answer: **Asynchronous programming** is a programming paradigm that lets a program perform tasks concurrently with other tasks, without blocking the main execution flow: operations such as network requests, file I/O, or database queries can start, run, and finish in the background while the rest of the program keeps responding. This non-blocking behaviour is what keeps an interface responsive instead of freezing while it waits. **AJAX** (Asynchronous JavaScript And XML) is the set of in-browser techniques that put this paradigm to work on the web: an event on the page triggers JavaScript to create an XMLHttpRequest object, which sends a request to the server in the background; the server processes it and sends back a response (in practice almost always JSON today, despite the "X" in the acronym); JavaScript reads that response and updates the DOM directly — all **without reloading the page**. This is the technology base for Single Page Applications (Gmail, Google Maps, Twitter): it reduces server load, avoids full page reloads, and produces the fluid, app-like experience users expect from modern dynamic web applications.

Question 13. What is the purpose of cookies? What are some common security threats associated with cookies? How can they be mitigated?

Model / correct answer: Purpose: a cookie is a small key-value pair a server asks the browser to store and send back on later requests. Cookies serve three purposes: **session management** (sign-in status, shopping-cart contents, scores), **personalization** (language, UI theme/preferences), and **tracking** (recording/analyzing user behaviour). **Threats** (named in the slides, after Park & Sandhu): **(1) Network threats** — cookies sent over an unencrypted channel can be eavesdropped on the wire. **(2) End-system threats** — cookies sit in clear text on the client's disk/memory and can be stolen, copied to another machine, or tampered with to impersonate the user (including reading them out of a browser/proxy cache, "cache sniffing"). **(3) Cookie harvesting** — an attacker impersonates a legitimate site to collect cookies users send it, or plants cross-site scripting (XSS) in a link (forum post, email) that, once clicked, hands over the victim's cookies. **Mitigations:** use HTTPS (SSL/TLS) and the Secure cookie attribute so cookies travel only over encrypted channels; use HttpOnly to block JavaScript access and blunt XSS-based theft; use SameSite to restrict cross-site sending and reduce CSRF risk; avoid putting sensitive data directly inside a cookie (store only an opaque session identifier and keep the real data server-side); and keep expiration short.

Question 14. Explain how the CSS cascade resolves a conflict when two or more rules target the same property on the same element. Be sure to cover specificity and the role of declaration order and !important.

Model / correct answer: When several rules could apply to the same element, the browser resolves the conflict with the **cascade algorithm**: (1) collect every rule whose selector matches the element; (2) compare each rule's **specificity** — computed as four counters, starting at 0 and adding 1 per element/pseudo-element selector, 10 per class/pseudo-class/attribute selector, 100 per ID selector, and 1000 for an inline style attribute — and apply whichever declaration has the **higher** specificity; (3) if two competing rules have **exactly equal** specificity, the one declared **later** in the stylesheet/document wins (source order is the final tie-breaker). On top of this, a declaration marked **!important** overrides every other rule for that property on that element **regardless of specificity**, which is why it should be used sparingly — it short-circuits the normal cascade and can make later debugging harder. One extra subtlety worth mentioning: values an element receives purely through **inheritance** from an ancestor carry no specificity at all (not even zero), so any rule that directly targets the element — however weak its specificity — will override an inherited value.

Question 15. Describe what happens, step by step, from the moment a Flask application receives an HTTP request to the moment it returns a rendered page, mentioning routing, the request object, and Jinja2 templating.

Model / correct answer: (1) The browser sends an HTTP request (e.g. a GET or POST) to a URL handled by the Flask app. (2) Flask's routing layer — built from `@app.route()` decorators — matches the requested path to a view function, extracting any variable segments declared in the route (e.g. `<username>` or a typed `<int:age>`) and passing them in as arguments. (3) Inside the matched view function, Flask exposes the global request object, which lets the function read the HTTP method used, query parameters, submitted form data (`request.form`), and cookies. (4) The view function runs whatever application logic it needs (e.g. validating login credentials against the database) and then calls `render_template(name, **context)`, handing a template file and a dictionary of data to the **Jinja2** engine. (5) Jinja2 substitutes every `{{ expression }}` with its value and executes every `{% statement %}` (loops, conditionals, and template inheritance via `{% extends %}/{% block %}`) against the supplied data, producing a final HTML string. (6) Flask wraps that HTML into an HTTP response (status 200 by default, unless the view says otherwise) and sends it back over the same connection; the browser receives it and renders the page.

Question 16. Compare HTTP Basic authentication and Digest authentication. Why is Digest considered more secure, and what is a replay attack in this context, with one way to prevent it?

Model / correct answer: Both schemes follow the same general HTTP authentication framework: the server issues a challenge and the client responds with credentials. **Basic authentication** (from HTTP/1.0) has the client send its `username:password` pair encoded in **Base64** inside the request header. Base64 is an **encoding**, not an encryption scheme — it is trivially reversible by anyone who intercepts it, so Basic authentication is roughly as insecure as sending the credentials in plain text unless the channel is otherwise secured (e.g. via HTTPS). **Digest authentication** avoids sending the password at all: the client instead computes a cryptographic **hash (digest)** derived from the password and sends only that digest; because a secure hash function is not invertible, the password itself never has to cross the network, which is why Digest is considered more secure than Basic. The remaining weakness is the **replay attack**: since the same password always produces the same digest, anyone who intercepts a valid digest could simply resend ("replay") it later and be authenticated without ever knowing the actual password. The standard prevention is to include a server-generated, single-use **nonce** ("number used once") in the challenge: the client hashes the password together with that nonce, so every digest is different and a captured digest cannot be reused once the nonce is no longer valid.

Mentor's closing note — If you scored well here without peeking, good — that is real signal. If you struggled on the open questions specifically, that is the part of the exam most students under-prepare for because it feels "less technical" than MC. It is not: graders want the same precision, just in your own sentences. Re-read the matching Part of this guide, then redo the open section from scratch in a week.